Robotics: Motor Parameter Identification

Week 5 Assignment 1

Module Title:	Robotics
Module Code:	CSC 413
Professors:	Aleksandr Kapitonov
Emails:	a.kapitonov@newuu.uz

Task Details

Description of the task	Every model-based method later in the course — localization, go-to-goal, filtering, pure pursuit — needs a model of the drive motors with the right numbers in it. In this assignment you identify the parameters of the TRIK DC motor from your own measurements. You build the data-collection program in TRIK Studio **visual programming**, drive the motor with a series of step inputs and with at least one complex test signal, process the data in Google Colab following the course notebook, and compare the identified model with the real motor. Special attention is paid to forward/backward asymmetry, zero drift and other measurement artifacts.
Type of assessment	Individual Work
The learning outcomes	CLO1 — Explain the principles of robotic kinematics and dynamics.

Assignment Task

Record the rotation of the robot's drive motors for a series of step inputs in both directions and for at least one complex input signal. From the step responses, estimate the steady-state speed ω_0 and the electromechanical time constant T_m at every PWM level, and plot both against PWM. Use the identified model to predict the motor's response to the complex signal and compare the prediction with the measurement. Analyse forward and backward rotation, zero drift and every other artifact you find in the data.

The robot program must be built with TRIK Studio visual (block) programming. This document gives the program only as pseudocode; translating it into blocks is part of the task. Data processing is done in Python in Google Colab, following Motor_identification.ipynb from the course repository. https://github.com/NewUU-engineering/Robotics-course/blob/main/05_Motor_parameter_identification/Motor_identification.ipynb

Steps and Time Allocations

1. Motor Model (15 minutes)

Neglecting the armature inductance, a lightly loaded DC motor is a first-order system in angular velocity (equation 5 of the course notebook):

$$d\omega/dt = U / (T_m\, k_e) - \omega / T_m \qquad d\theta/dt = \omega$$

For a constant voltage applied at $t = 0$ from rest, the angle is

$$\theta(t) = \omega_{ss} \cdot (t - T_m + T_m \cdot \exp(-t / T_m)), \qquad \omega_{ss} = U / k_e$$

Symbol	Meaning	Unit
U	Voltage applied to the motor, U = PWM/100 · $U_{battery}$	V

Symbol	Meaning	Unit
θ, ω	Shaft angle and angular velocity	rad, rad/s
ω_{ss}	Steady-state speed reached for a given U	rad/s
T_m	Electromechanical time constant: time to reach 63 % of ω_{ss}	s
k_e	Back-EMF constant: voltage needed per rad/s of speed	V·s/rad

Requirements:

- Explain each parameter and its unit in your own words.
- Show how $\theta(t)$ above follows from the differential equation for a step input.
- State the assumptions of the model: no inductance, linear viscous friction, constant load, constant battery voltage within a run. Each of them will be tested by your data.

2. Test Signals (10 minutes)

A single step at one level shows only one operating point. Identification needs the motor excited at **many speeds, in both directions, and at different rates of change**. You must use:

Signal	What it reveals
Step series	ω_{ss} and T_m at each PWM level; dead zone; forward/backward asymmetry; saturation
Square wave	Repeated accelerations and direction reversals; gear backlash; T_m in both directions within one run
Sawtooth	Slow sweep through all speeds followed by a sudden reversal; dead-zone crossing
Triangle	Symmetric slow sweep up and down; hysteresis between rising and falling input
Chirp (swept sine)	Frequency response: amplitude falls and lag grows as frequency increases; direct check of T_m
Other (e.g. PRBS — random ±A switching)	Rich excitation for identification; justify your choice in the report

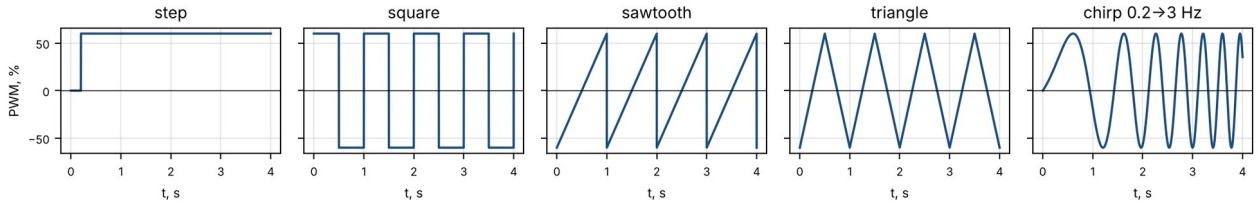


Figure 1. Examples of test signals (amplitude 60 %, period 1 s, chirp 0.2–3 Hz over 4 s).

Formulas for the complex signals, with t in seconds, amplitude A (% PWM), period P, and $\mathrm{frac}(x) = x - \mathrm{floor}(x)$:

Signal	u(t), % PWM
Square	A if $\mathrm{frac}(t/P) < 0.5$, otherwise $-A$
Sawtooth	$A \cdot (2 \cdot \mathrm{frac}(t/P) - 1)$
Triangle	$A \cdot (1 - 4 \cdot \lvert\mathrm{frac}(t/P) - 0.5\rvert)$
Chirp	$A \cdot \sin(2\pi \cdot (f_0\, t + (f_1 - f_0)\, t^2 / (2T)))$ for $0 \le t \le T$

Design rules:

- The period should be several times T_m (expect roughly 0.05–0.2 s). Much shorter, and the motor barely moves; much longer, and you see only steady state.

- A chirp should start well below $1/(2\pi T_m)$ and end well above it.
- The signal must cross zero, so that both directions and the dead zone appear in the same run.
- Use the full PWM range in the step series, but keep the complex signal amplitude ≤ 80 % so the model is not tested only in saturation.

3. Data Collection Program in TRIK Studio Visual Programming (45 minutes)

Build two programs from blocks. The pseudocode below names the TRIK Studio blocks in square brackets. Expressions inside [Function] blocks are written in the TRIK Studio expression language; check the exact syntax and available functions (time(), sin, floor, abs) in the TRIK help. If a function is missing, build it from blocks or simple arithmetic. **Do not use JavaScript or Python text programs.**

Program A — step series (motor_steps.qrs). One file for the whole series; every line stores the angle, the time and the commanded PWM, so the processing script can split the record by level.

```
[Remove File]       "steps_M4.txt"
[Function]          level = −100                    // first PWM level, %

// --- zero-drift check: motor off, encoder logged for 3 s -----------
[Function]          t0 = time()
[Loop] while  time() - t0 < 3000:
   [Function]       theta = encoder4 ; t = time()
   [Write To File]  "steps_M4.txt" <- theta .. "," .. t .. "," .. 0 .. "\n"
   [Timer]          5 ms

// --- step series ---------------------------------------------------
[Loop] while  level <= 100:
   [Condition]      if level != 0:
      [Function]       t0 = time()
      [Motor]          M4 power = level            // the step
      [Loop] while  time() - t0 < 2000:            // 2 s with motor on
         [Function]       theta = encoder4 ; t = time()
         [Write To File]  "steps_M4.txt" <- theta .. "," .. t .. "," .. level .. "\n"
         [Timer]          5 ms
      [Motor]          M4 power = 0
      [Loop] while  time() - t0 < 3000:            // 1 s coasting, still logged
         [Function]       theta = encoder4 ; t = time()
         [Write To File]  "steps_M4.txt" <- theta .. "," .. t .. "," .. 0 .. "\n"
         [Timer]          5 ms
   [Function]       level = level + 10
[Motor]             M4 stop
```

Program B — complex signal (motor_signal.qrs). Same logging, but the PWM is recalculated in every cycle from the chosen formula.

```
[Remove File]       "signal_M4.txt"
[Function]          A = 60 ; P = 1000 ; T = 8000     // %, ms, ms
[Function]          t0 = time()
[Loop] while  time() - t0 < T:
   [Function]       tau = time() - t0
                    u   = SIGNAL(tau)               // formula from section 2
                    u   = round(u)
   [Motor]          M4 power = u
   [Function]       theta = encoder4 ; t = time()
   [Write To File]  "signal_M4.txt" <- theta .. "," .. t .. "," .. u .. "\n"
   [Timer]          5 ms
[Motor]             M4 power = 0
[Timer]             1000 ms                         // log coasting here as in Program A
[Motor]             M4 stop
```

Requirements:

- Run both programs on **both drive motors** (left and right). Change the motor and encoder port and the file name; keep everything else identical.

- Lift the robot so the wheels spin freely, and say so in the report. Optional: repeat one step level with the robot on the floor to see the effect of load.
- Write down the battery voltage before and after each series (TRIK display or a voltage reading block). You need it to convert PWM to volts.
- Keep the [Timer] delay fixed. Note the real sampling interval you obtain: it will be longer than 5 ms because writing to a file takes time.
- Run Program A twice on the same motor to check repeatability.
- Download the files from the robot without editing them.

4. Data Processing in Colab (45 minutes)

Follow `Motor_identification.ipynb` (course repository, folder `05_Motor_parameter_identification`) and extend it as described below.

4.1 Loading and SI conversion. Load the file with `np.loadtxt(..., delimiter=',')`. Note that the column order in your files is *angle*, *time*, *PWM*: adapt the notebook's loading line accordingly. Convert time to seconds from the start of each segment and the angle to radians, as in the notebook.

4.2 Step responses. Split the step file into segments by the PWM column. For every segment subtract the initial angle and initial time, then fit the notebook's model with `optimize.curve_fit`:

```
def model(t, speed, T_m):
    return speed * t - speed * T_m + speed * T_m * np.exp(-t / T_m)

popt, pcov = optimize.curve_fit(model, t_seg, theta_seg, p0=[8, 0.1])
speed, T_m = popt                    # omega_ss [rad/s], T_m [s]
err = np.sqrt(np.diag(pcov))         # standard errors of the fit
```

Collect ω_{ss}, T_m and their standard errors for every PWM level in a table, for each motor.

4.3 Speed and time constant versus PWM. Produce the two required plots:

- **Speed from PWM:** ω_{ss} against PWM from −100 % to +100 % at least 10% step, forward and backward in different colours, both motors on the same axes. Fit a straight line to the linear part of each direction separately and mark the dead zone.
- **Time constant from PWM:** T_m with error bars against PWM, same layout. Discuss whether T_m is really constant, as the model assumes.

Convert PWM to voltage with $U = \text{PWM}/100 \cdot U_{battery}$ and compute $k_e = U / \omega_{ss}$ from the slope of the linear part, for each direction.

4.4 Complex signal: model versus real motor. Use the identified T_m and k_e to simulate the motor with the recorded input. In the notebook, `u` is a constant; make it a function of time by interpolating the logged PWM:

```
def motor(state, t):
    speed, angle = state
    u = np.interp(t, t_log, U_log)          # recorded input, converted to volts
    return np.array([-speed / T_m + u / (T_m * k_e), speed])

y = odeint(motor, [0, 0], t_log)            # y[:,1] = simulated angle
```

Add a dead zone to the model if your step data show one, for example by setting `u = 0` when $|U|$ is below the identified threshold. Compare the model with and without it.

Plot on the same axes: the measured and simulated angle, the measured and simulated speed, and the input PWM on a second axis. Obtain the measured speed by numerical differentiation of the angle (`np.gradient`) and smooth it with a moving average; state the window. Report RMSE and fit percentage:

$$\text{fit} = 100 \cdot (1 - \|y - \hat{y}\| / \|y - \text{mean}(y)\|)$$

For a chirp, also plot the ratio of speed amplitude to input amplitude against frequency, and compare it with the model's prediction $1 / (k_e \sqrt{(1 + (2\pi f\, T_m)^2)})$.

5. Artifact Analysis (25 minutes)

Real data are never as clean as the model. Look for each of the following in your records, show it on a plot or with a number, and explain its cause:

Artifact	Where to look	What to report
Forward vs backward asymmetry	Speed-from-PWM plot; step responses at ±level	Difference in slope, dead zone and T_m between directions, in %
Zero drift	The 3 s zero-PWM block at the start; coasting segments; final angle after the series	Does the encoder change with the motor off? Does the angle return to a consistent zero? Time offset of `time()` at the start
Dead zone	Low PWM levels; zero crossings of complex signals	PWM threshold below which the motor does not move, per direction
Saturation	High PWM levels	Level above which ω_{ss} stops increasing linearly
Coasting after power-off	1 s after each step	How far the wheel turns after the motor is switched off; compare with T_m
Gear backlash	Direction reversals in square or triangle signal	Delay or flat region in the angle at each reversal
Sampling jitter	Histogram of Δt between samples	Mean, minimum, maximum interval; effect on the speed estimate
Battery sag	Voltage before and after; the same level early and late in the series	Change in $U_{battery}$ and its effect on ω_{ss}
Encoder quantisation and differentiation noise	Speed estimated by `np.gradient`	Noise level with and without smoothing
Motor-to-motor spread	Left vs right	Differences in ω_{ss}, T_m and k_e

Report Requirements

The report is the Colab notebook itself (Markdown cells between the code). It must contain:

1. Estimated values in this form, one table per motor:

Direction	Dead zone, % PWM	Slope, rad/s per % PWM	k_e, V·s/rad	Mean T_m, s	$U_{battery}$, V
Forward					
Backward					

2. Required plots, each with a short caption:

1. Step responses $\theta(t)$ for different PWM levels with the fitted model overlaid.
2. Speed from PWM: ω_0(PWM), forward and backward, both motors, with linear fits and dead zone marked.
3. T_m from PWM, with error bars.
4. Complex signal: measured vs simulated angle and speed, with the input PWM.
5. Complex signal: error (measured − simulated) over time.
6. Artifact evidence: zero-drift record and at least one more artifact from section 5.

3. Analysis (one paragraph each):

- How well does the first-order model with constant T_m and k_e describe the motor? Give the fit percentage on the complex signal and point to where the error is largest.
- Forward vs backward rotation: are the two directions the same motor? Should a controller use one set of parameters or two?
- Which artifacts did you find, how large are they, and which of them matter for odometry next week?
- Which complex signal did you choose and what did it show that the step series did not?

Resources and References

- Course repository, week 5: github.com/NewUU-engineering/Robotics-course → 05_Motor_parameter_identification — notebook Motor_identification.ipynb, example program Motor_identification.qrs, reference measurements in src/.
- ETH Zürich AMOD 2020 — Modeling lecture (DC motor and differential drive).
- Corke, P. *Robotics, Vision and Control*, Chapter 4 (Mobile Robots).
- SciPy documentation: scipy.optimize.curve_fit, scipy.integrate.odeint.
- TRIK Studio help — visual programming blocks and expression language: help.trikset.com/en

Deliverables

Create CS413 - Robotics/Lab5/ containing:

1. **motor_steps.qrs** and **motor_signal.qrs** — the TRIK Studio visual programs.
2. **motor_data/** — all raw files exactly as the robot wrote them, plus a battery.txt with the voltages you noted.
3. **robotics_lab5_[studentID].ipynb** — Colab notebook with processing, plots and the report. It must run from top to bottom on the files in motor_data/.

Evaluation Criteria

Level	Description
Excellent (90–100 %)	Clean visual programs; step series in both directions on both motors, repeated; at least one complex signal; speed-from-PWM and T_m-from-PWM plots with fits and error bars; model with dead zone reproduces the complex signal with fit > 85 %; most artifacts from section 5 identified and quantified; analysis connects each discrepancy to a physical cause.
Good (75–89 %)	All required runs and plots present; parameters for both directions; model–measurement comparison with metrics; several artifacts analysed; discussion sensible but partly generic.
Satisfactory (60–74 %)	Step series in one direction only, or one motor only; plots present but without fits or errors; complex signal recorded but comparison only qualitative; few artifacts discussed.
Needs Improvement (< 60 %)	Program written as text instead of blocks; raw data missing or edited; notebook does not run; no comparison of model and real motor; parameters without units.

Submission

Given date: Week 5

Submission deadline: before the Week 8, 23:59

Submission form: https://docs.google.com/forms/d/e/1FAIpQLSchchoZpAJJ40YK4PbSLPuiZs6egEaAZ-E0rieg1XZllkc6FA/viewform?usp=dialog

AI Tool Usage Policy

✓ **Allowed:**

- Using AI for code explanation and debugging
- Generating boilerplate code structures
- Understanding mathematical concepts
- Optimizing existing algorithms

✗ **Not Allowed:**

- Direct copy-paste of entire solutions
- Using AI to write analysis without understanding
- Submitting AI-generated reports without modification
- Generating, altering or “cleaning” measurement data by hand